

ARGOS - Prompt Mestre de Implementacao

Arquitetura de IA local + fallback para nuvem + classificacao de intencao + tool calling + voz + automacoes

Como usar: entregue este documento inteiro ao Claude Code no repositorio atual do Argos. O objetivo nao e reescrever o aplicativo. Primeiro ele deve auditar o que ja existe, preservar o que funciona e integrar a nova arquitetura de ponta a ponta.

PROMPT PARA O CLAUDE CODE

Voce esta trabalhando no projeto ARGOS, um assistente pessoal por voz para Android. Sua tarefa e analisar o projeto existente e implementar, de forma incremental, uma arquitetura hibrida capaz de executar comandos simples com baixa latencia no proprio dispositivo e usar um LLM local ou a nuvem apenas quando necessario.

REGRA PRINCIPAL: NAO COMECE CODIFICANDO.

Primeiro faca uma auditoria completa do repositorio. Identifique o que ja existe, o que esta funcional, o que esta parcialmente implementado, o que pode ser reutilizado e quais mudancas seriam destrutivas. Nao duplique componentes existentes. Nao substitua uma integracao funcional apenas porque existe outra biblioteca que voce prefere.

Antes de qualquer alteracao, produza:

1. mapa da arquitetura atual;
2. fluxo atual de voz/comandos;
3. lista de componentes existentes;
4. integracoes externas;
5. sistema atual de wake word;
6. STT atual;
7. TTS atual;
8. backend/cloud atual;
9. sistema de automacoes;
10. sistema de dispositivos/casa inteligente;
11. memoria/contexto;
12. permissoes Android;
13. servicos em foreground/background;
14. notificacoes;
15. banco/local storage;
16. APIs e tool calls existentes;
17. pontos de risco;
18. plano incremental de implementacao.

Somente depois dessa auditoria, implemente.

1. OBJETIVO DE EXPERIENCIA

O usuario deve perceber apenas um unico Argos. Ele nao deve precisar saber se uma resposta veio de regra deterministica, classificador, LLM local ou nuvem.

Fluxo desejado:

Usuario fala -> Argos detecta -> transcreve -> entende -> decide a rota -> valida -> executa ferramenta -> confirma resultado -> responde por voz.

Exemplo:

"Argos, apaga a luz da sala."

A resposta ideal deve ser praticamente imediata:

"Pronto."

Para comandos simples, evite chamar um LLM se uma solucao deterministica ou classificador leve for suficiente.

2. ARQUITETURA EM CAMADAS

Implemente ou adapte a arquitetura para possuir as seguintes camadas independentes:

- A. Wake Word / ativacao
- B. Captura de audio
- C. Voice Activity Detection, se aplicavel
- D. Speech-to-Text
- E. Normalizacao da entrada
- F. Fast Path deterministico
- G. Classificador de intencao
- H. Extracao de entidades/slots
- I. Confidence + Coverage Gate
- J. LLM local
- K. Router para cloud
- L. Tool Registry
- M. Policy/Permission Engine
- N. Executor
- O. Verification Layer
- P. Memoria/contexto
- Q. Text-to-Speech
- R. Telemetria local de latencia e erros

Cada camada deve possuir interface clara e poder ser substituida sem reescrever o restante do sistema.

3. FAST PATH

Antes de usar um LLM, detectar comandos simples e frequentes.

Exemplos:

- apagar/acender luz;
- consultar estado de dispositivo;
- ligar/desligar ar-condicionado;
- abrir uma tela do Argos;
- executar uma automacao previamente criada;
- consultar uma informacao local simples.

O Fast Path deve ser extremamente rapido e previsivel. Nao use regex fragil como unica estrategia se o projeto ja possuir uma solucao melhor.

4. CLASSIFICADOR DE INTENCAO

Criar uma representacao estruturada para a interpretacao.

Exemplo conceitual:

```
{
  "intent": "device.turn_off",
  "entities": {
    "device_type": "light",
    "room": "living_room"
  },
  "confidence": 0.97,
  "coverage": 1.0,
  "ambiguities": [],
  "unparsed_segments": []
}
```

A frase NAO deve ser considerada compreendida apenas porque uma parte dela corresponde a uma intencao.

Exemplo:

"Apaga a luz da sala mas deixa a do corredor ligada."

O sistema precisa representar a frase inteira ou encaminha-la para uma camada mais inteligente.

Nunca descarte silenciosamente trechos nao compreendidos.

5. CONFIDENCE E COVERAGE

A decisao de executar nao deve depender do numero de caracteres nem do tamanho da frase.

Avaliar pelo menos:

- confidence: quao confiante esta na intencao;
- coverage: quanto da solicitacao foi explicado;
- required slots: todos os parametros obrigatorios existem?;
- ambiguity: existe mais de uma interpretacao plausivel?;
- action risk: qual o impacto da acao?;
- context dependency: a frase depende de conversa anterior?;
- multi-intent: existem varias acoes na mesma frase?

Defina thresholds configuraveis, nao numeros espalhados pelo codigo.

Exemplo inicial, a ser calibrado por testes:

alta confianca + cobertura completa + acao de baixo risco -> executar;

incerteza moderada -> LLM local;

alta complexidade, necessidade externa ou baixa confianca -> cloud;

ambiguidade que nao pode ser resolvida -> perguntar ao usuario.

O sistema jamais deve executar uma interpretacao parcial como se fosse completa.

6. LLM LOCAL

Investigar qual runtime/modelo local e realmente adequado ao Android e ao hardware alvo atual.

Nao escolha um modelo antes de medir:

- tamanho em disco;
- RAM real durante inferencia;
- tempo para carregar;
- time-to-first-token;
- tokens/segundo;
- consumo de bateria;
- aquecimento;
- estabilidade;
- suporte ao conjunto de ABIs do app;
- compatibilidade com aparelhos fracos;
- licenca para distribuicao comercial.

O LLM local deve ser pequeno e ter funcao clara:

- interpretar linguagem mais flexivel;
- resolver referencias simples;
- transformar pedido em chamadas estruturadas;
- produzir respostas curtas;
- evitar cloud quando for razoavel.

Ele NAO recebe permissao irrestrita sobre Android ou dispositivos.

Ele solicita ferramentas.

7. FALLBACK PARA CLOUD

Criar um Router central.

Rotas possiveis:
FAST_PATH
INTENT_LOCAL
LLM_LOCAL
CLOUD
CLARIFICATION

Enviar para cloud quando, por exemplo:

- exigir conhecimento atualizado;
- exigir pesquisa/web;
- exigir raciocinio alem da capacidade local;
- o modelo local nao tiver confianca;
- houver contexto remoto necessario;
- o dispositivo nao suportar adequadamente inferencia local.

A transicao deve ser transparente para o usuario.

Se a internet estiver indisponivel, informar a limitacao apenas quando ela realmente impedir a tarefa. Comandos locais devem continuar funcionando offline sempre que tecnicamente possivel.

8. TOOL CALLING

O modelo nunca deve controlar diretamente componentes sensiveis.

Criar/manter um Tool Registry com contratos tipados.

Exemplos conceituais:

turnLightOn(deviceId)
turnLightOff(deviceId)
setAirConditioner(deviceId, mode, temperature)
getDeviceState(deviceId)
createAutomation(rule)
enableAutomation(id)
disableAutomation(id)
getHouseOccupancy()
openArgosScreen(route)
getLocalMemory(query)

Cada ferramenta deve possuir:

- nome;
- descricao;
- schema de entrada;
- schema de saida;
- nivel de risco;
- permissoes necessarias;
- validacao;
- timeout;
- tratamento de erro;
- idempotencia quando aplicavel.

LLM -> solicita ToolCall.

Argos -> valida.

Executor -> executa.

Verification Layer -> confirma resultado.

Somente entao Argos afirma que fez a acao.

Nunca responder "desliguei" antes de receber confirmacao real da integracao/dispositivo.

9. AUTOMACOES E AUTONOMIA

Preservar a filosofia do Argos: ele pode tornar-se proativo e executar automaticamente tarefas que o usuário explicitamente autorizou.

Fluxo:

1. detectar um padrão;
2. sugerir automação;
3. usuário aceita explicitamente;
4. mostrar exatamente o que passara a acontecer;
5. registrar consentimento;
6. executar automaticamente nas condições definidas;
7. notificar de forma adequada quando relevante;
8. permitir desfazer/desativar facilmente.

Exemplo:

O usuário repetidamente sai de casa e desliga luzes/ar.

Argos pode sugerir:

"Quer que eu faça isso automaticamente quando todos saírem?"

Se aceito:

"Quando nenhum membro da casa estiver presente, desligar luzes X e ar Y."

A automação precisa considerar contexto, como outros moradores presentes.

Toda automação deve ter:

- trigger;
- conditions;
- actions;
- exceptions;
- enabled;
- consent/version;
- createdAt;
- lastRun;
- execution log;
- opção de desativação.

10. SEGURANÇA E PERMISSÕES

Antes de executar qualquer ToolCall:

- verificar autenticação;
- verificar permissão Android;
- verificar permissão do usuário;
- verificar permissão da residência/conta;
- verificar dispositivo alvo;
- validar argumentos;
- aplicar política de risco;
- impedir comandos inventados pelo modelo;
- proteger contra prompt injection oriunda de dados externos.

Ações irreversíveis, financeiras, acesso físico sensível ou outras categorias de alto impacto devem exigir políticas mais fortes do que apagar uma luz.

Não armazenar segredo/API key sensível diretamente no APK.

Use armazenamento seguro e arquitetura apropriada ao segredo.

11. STT E TTS

Auditar primeiro o que o Argos já usa.

Objetivo:

voz -> texto -> decisão -> execução -> texto -> voz.

Avaliar opções locais e cloud sem trocar tecnologia funcional sem justificativa.

Para TTS:

- respostas de comando devem ser curtas;
- iniciar fala rapidamente;
- se possível, permitir streaming em respostas longas;
- evitar gerar texto excessivo para confirmações simples.

Exemplos:

"Pronto."

"Desliguei."

"O ar da sala está em 22 graus."

Não responder com um parágrafo quando uma palavra resolve.

12. HARDWARE ADAPTATIVO

O Argos não deve assumir que todo Android consegue rodar o mesmo modelo.

Criar DeviceCapabilityProfile avaliando, conforme APIs disponíveis:

- RAM;
- memória disponível;
- CPU/ABI;
- aceleradores disponíveis;
- armazenamento;
- versão Android;
- benchmark curto opcional;
- comportamento térmico/performance observado.

A partir disso selecionar estratégia, por exemplo:

CLOUD_FIRST

HYBRID_LIGHT

HYBRID_LOCAL

Não bloquear celulares fracos se o Argos puder funcionar via cloud.

O aparelho fraco de teste deve ser tratado como baseline de compatibilidade, mas não assuma que, se funcionar nele, automaticamente funcionará perfeitamente em todos os Android. Fragmentação, fabricante, RAM, ABI e políticas de background variam.

13. DOWNLOAD DO MODELO

Não obrigar necessariamente que um modelo grande esteja embutido no APK.

Comparar:

- A. modelo dentro do pacote;
- B. asset pack;
- C. download após instalação;
- D. download opcional conforme capacidade;
- E. modelos diferentes por perfil de hardware.

Implementar:

- download resiliente;
- checksum;
- versionamento;
- rollback;
- espaço necessário antes do download;
- progresso;
- cancelamento;
- atualização segura;
- exclusão/liberação de espaço.

O app deve continuar utilizável mesmo se o modelo local não estiver instalado.

14. LATENCIA E BENCHMARK

Instrumentar cada etapa separadamente.

Registrar:

wake detection
audio capture end
STT duration
routing duration
intent duration
local LLM load
local LLM TTFT
tool call duration
device acknowledgement
cloud round trip
TTS start
total end-to-end

Criar uma tela/log de desenvolvimento para comparar:

TESTE A - caminho atual/cloud
TESTE B - fast path
TESTE C - classificador
TESTE D - LLM local

Executar varias repeticoes do mesmo comando.

Nao concluir que local e mais rapido sem medir.

Meta: comandos frequentes devem utilizar o caminho mais rapido que mantenha confiabilidade.

15. TESTE INICIAL DE PONTA A PONTA

Antes de implementar dezenas de ferramentas, provar uma vertical completa com UMA luz real ja integrada ao Argos.

Teste:

"Argos, apaga a luz [nome real]."

Fluxo:

wake word -> STT -> router -> intent -> entity resolution -> permission -> tool call -> integracao real -> confirmacao do estado -> TTS.

Depois testar:

"Acende de novo."

"Apaga a luz da sala."

"Qual luz esta ligada?"

"Apaga a luz da sala, mas nao mexe na do corredor."

"Quando eu sair, apaga essa luz."

"Se nao tiver mais ninguem em casa, apaga tudo."

Testar erros:

dispositivo offline;
nome ambiguo;
STT incorreto;
sem internet;
modelo local indisponivel;
permissao negada;
timeout;
acao falha apos ToolCall.

16. TESTES AUTOMATIZADOS

Criar testes para:

- roteamento;
- intents;
- entities;
- confidence thresholds;
- coverage;
- frases ambíguas;
- multi-intent;
- contexto;
- tool schema;
- permission engine;
- executor;
- falhas;
- automações;
- regressão.

Criar dataset local de utterances em português, incluindo fala informal e variações.

Exemplos:

"apaga a luz"

"desliga a luz da sala"

"pode apagar a sala pra mim"

"deixa tudo escuro aqui"

"apaga tudo menos o corredor"

"quando eu sair daqui desliga o ar"

"se minha mulher ainda estiver em casa não desliga nada"

Para cada utterance registrar a interpretação esperada.

17. OBSERVABILIDADE E PRIVACIDADE

Logs de desenvolvimento devem permitir diagnosticar por que uma rota foi escolhida sem registrar desnecessariamente dados privados.

Quando possível registrar metadados:

route=INTENT_LOCAL

intent=device.turn_off

confidence=0.96

coverage=1.0

latency=...

tool=...

result=success

Evitar colocar áudio, transcrição privada ou tokens sensíveis em logs de produção sem política explícita.

18. PLANO DE IMPLEMENTAÇÃO

Trabalhe em fases e mantenha o app executável ao final de cada fase.

FASE 0 - auditoria

FASE 1 - interfaces e Router sem alterar comportamento

FASE 2 - Tool Registry envolvendo as funções existentes

FASE 3 - Fast Path + intent classifier

FASE 4 - confidence/coverage gate

FASE 5 - teste real com uma luz

FASE 6 - STT/TTS otimizado

FASE 7 - LLM local em feature flag

FASE 8 - fallback cloud

FASE 9 - DeviceCapabilityProfile + modelo/download adaptativo

FASE 10 - automações proativas autorizadas

FASE 11 - benchmarks, bateria, estabilidade e regressao

A cada fase:

- compilar;
- rodar testes;
- testar no dispositivo;
- documentar mudancas;
- nao prosseguir se houver regressao critica.

19. CRITERIOS DE ACEITE

A implementacao so pode ser considerada pronta quando:

1. o Argos existente continua funcional;
2. comandos simples podem evitar cloud;
3. nenhuma acao e executada a partir de interpretacao parcial;
4. ha fallback transparente;
5. LLM nao possui acesso direto irrestrito;
6. ToolCalls sao validadas;
7. a resposta falada ocorre apenas depois do resultado real quando afirma uma acao;
8. comandos locais continuam funcionando sem internet quando possivel;
9. celulares sem capacidade local continuam funcionando via cloud;
10. modelo local e opcional/adaptativo;
11. automacoes exigem autorizacao explicita antes de se tornarem autonomas;
12. usuario pode desativar automacoes;
13. existe telemetria de latencia;
14. existe suite de testes;
15. existe comparacao objetiva local vs cloud;
16. existe documentacao da arquitetura final.

20. INSTRUCAO FINAL AO CLAUDE CODE

Agora:

1. NAO altere codigo ainda.
2. Analise todo o repositorio.
3. Mostre o mapa do sistema atual.
4. Relacione cada componente existente com a arquitetura proposta.
5. Identifique o que ja esta pronto.
6. Identifique o que falta.
7. Identifique incompatibilidades e riscos.
8. Proponha o menor plano incremental possivel.
9. Mostre quais arquivos pretende criar/modificar e por que.
10. Aguarde aprovacao antes das mudancas estruturais de alto impacto.

Quando implementar, prefira adaptar e conectar o que ja existe a criar sistemas paralelos.

Objetivo final: o Argos deve ter uma unica experiencia de assistente, mas internamente escolher automaticamente o caminho mais rapido, barato, privado e confiavel para cada solicitacao.